

Practica 01

Integrantes del equipo:

Pérez Gaytán Ángel Gabriel 320052947

Gustavo Emir Jimenez Cruz 319260603

Jose Victor Hernandez Rodriguez 422033332

Versión de java usada: openjdk 21.0.10 2026-01-20

Anotaciones:

Jose Victor Hernandez Rodriguez no colaboró en absolutamente nada en esta práctica, por lo que lo consideramos fuera del equipo.

El objetivo central era implementar ambos patrones para modelar un juego de peleas. Este código lo logra de manera robusta y orientada a objetos:

Patrón Observer (Transmisión del Combate). El requerimiento indicaba que los espectadores debían unirse a una sesión y recibir actualizaciones detalladas de lo que estaba pasando.

- **Sujeto (Interfaz y Clase Combate):** La clase Combate actúa como el Sujeto Concreto. Mantiene una lista de observadores y su estado interno (estadoSujeto). Cada vez que ocurre un evento en la pelea (alguien ataca, cambia de forma o es derrotado), se llama a setEstado(), lo cual dispara automáticamente el método notificar().
- **Observador (Interfaz y Clase Espectador):** La clase Espectador representa a los usuarios simulados. Al instanciarse, se registran a sí mismos en la arena (sujeto.registrar(this)). Cuando el combate notifica un cambio, se ejecuta el método actualizar(), permitiendo que cada espectador guarde el evento exacto en su bitácora. Al finalizar, generan un archivo personalizado según el ID y el ganador.

Patrón Strategy (Cambio Dinámico de Poderes). El documento estipula que las acciones de los personajes cambian con respecto al poder consumido y que las opciones deben estar limitadas a sus propias franquicias.

- **Estrategia (Interfaz EstrategiaPoder):** Define un contrato común (aplicarPoder()) que todas las transformaciones deben cumplir. Para respetar el balance del juego, se crearon sub-interfaces vacías (como ObjetoEspecialKorby) que actúan como "etiquetas" para evitar que Korby obtenga poderes de MeganMan.
- **Estrategias Concretas (Las clases de Poderes):** Clases como BolaFuego o ArmaElectrica encapsulan algoritmos específicos (en este caso, los nuevos stats de ataque y defensa). Al ser llamadas, alteran los valores del personaje.
- **Contexto (Clases Korby, MeganMan, Dittu):** Los personajes mantienen una referencia a su estrategia permitida. Cuando obtienen un objeto, delegan la modificación de sus estadísticas al objeto mismo a través de objeto.aplicarPoder(this). Esto cumple el principio Open/Closed; si el día de mañana quieres agregar 10 poderes nuevos, no tienes que modificar el código interno de los personajes.